

CAPÍTULO 3

Arrays, Strings y Métodos

Agrupar datos, dominar el texto y organizar tu código en piezas reutilizables.



En este capítulo aprenderás a...

- ✓ Crear y recorrer arrays de una y varias dimensiones.
- ✓ Manejar texto con String y entender por qué es inmutable.
- ✓ Usar StringBuilder/StringBuffer para construir texto de forma eficiente.
- ✓ Definir tus propios métodos con parámetros y valores de retorno.
- ✓ Sobrecargar métodos (overloading) y escribir tu primera recursión.

1 Arrays: colecciones de tamaño fijo

Hasta ahora cada variable guardaba un único valor. Pero, ¿y si necesitas las notas de 30 estudiantes? Crear 30 variables sería absurdo. Un array es una colección de valores del MISMO tipo, guardados juntos bajo un solo nombre y accesibles por su posición (índice).

Un array: cajas numeradas desde 0

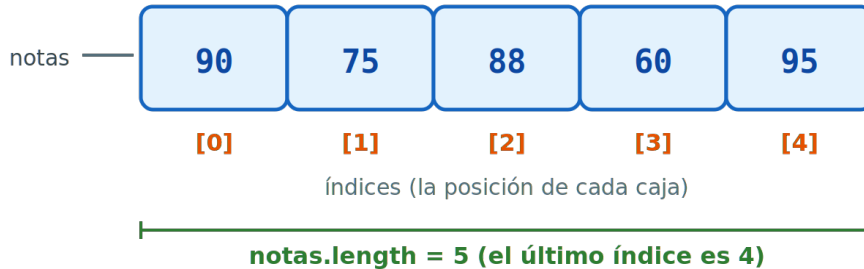


Figura 3.1 — Un array de 5 enteros. Los índices empiezan en 0.

1.1 Declarar y crear un array

Para declarar un array indicas el tipo seguido de corchetes []. Para crearlo con un tamaño usas `new`; el tamaño es fijo y no cambia después.

```
Arrays1.java

1 int[] notas = new int[5];      // array de 5 enteros (valen 0 por defecto)
2 String[] nombres = new String[3];
3 double[] precios;             // declarado, aún sin crear
4 precios = new double[4];       // ahora sí, creado con espacio para 4
```

Valores por defecto

Al crear un array con `new`, Java rellena cada casilla con un valor por defecto: 0 para números, `false` para boolean, `"\u0000"` para char y `null` para objetos (como `String`). Nunca queda “basura”.

1.2 Inicializar con valores

Si ya conoces los valores, puedes crear y llenar el array en una sola línea con llaves {}:

```
Arrays2.java

1 int[] notas = {90, 75, 88, 60, 95}; // forma literal: tamaño automático
2
3 String[] dias = {"Lun", "Mar", "Mié", "Jue", "Vie"};
4
5 // Equivale a crear y luego asignar uno por uno:
6 int[] otros = new int[3];
7 otros[0] = 10; otros[1] = 20; otros[2] = 30;
```

1.3 Acceder, modificar y medir

Usas el índice entre corchetes para leer o cambiar un valor. La propiedad `.length` te dice cuántos elementos hay (¡no es un método, no lleva paréntesis!).

```
Arrays3.java

1 int[] notas = {90, 75, 88, 60, 95};
2
3 System.out.println(notas[0]);      // 90 (primer elemento)
4 System.out.println(notas[4]);      // 95 (último elemento)
5
6 notas[1] = 80;                     // modificar la posición 1
7 System.out.println(notas.length); // 5 (cantidad de elementos)
```

⚠ El error más famoso: `ArrayIndexOutOfBoundsException`

Si un array tiene 5 elementos, sus índices válidos van de 0 a 4. Acceder a `notas[5]` lanza `ArrayIndexOutOfBoundsException`, porque esa caja no existe. Recuerda: el último índice siempre es `length - 1`.

1.4 Recorrer un array

Para procesar todos los elementos usas un bucle. Con `for` tienes el índice (útil si necesitas la posición); con `for-each` el código queda más limpio cuando solo importan los valores.

```
Recorrer.java

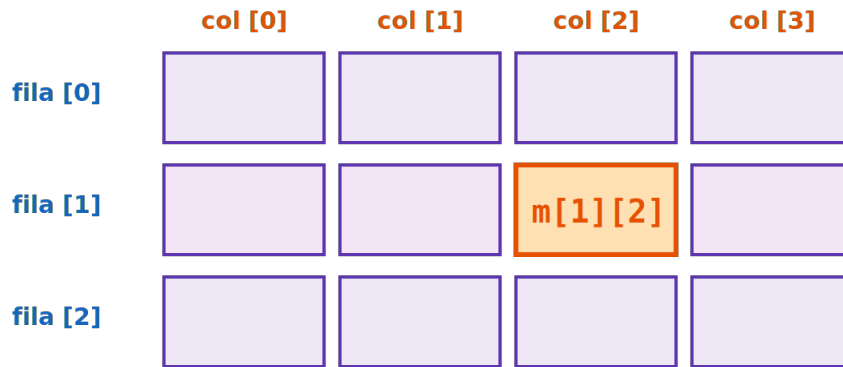
1 int[] notas = {90, 75, 88, 60, 95};
2
3 // Con for clásico (tienes el índice i):
4 for (int i = 0; i < notas.length; i++) {
5     System.out.println("Nota " + i + ": " + notas[i]);
6 }
7
8 // Con for-each (más limpio si no necesitas el índice):
9 int suma = 0;
10 for (int nota : notas) {
11     suma += nota;
12 }
13 System.out.println("Promedio: " + (suma / notas.length));
```

1.5 Arrays multidimensionales (matrices)

Un array puede contener otros arrays. El caso más común es el array 2D o matriz: una tabla con filas y columnas, perfecta para tableros, cuadrículas u hojas de cálculo.

Array 2D: una tabla de filas y columnas

`int[][] m = new int[3][4]; // 3 filas, 4 columnas`



Cada celda se localiza con dos índices: `m[fila][columna]`

Figura 3.2 — Una matriz se indexa con dos corchetes: `m[fila][columna]`.

```
Matriz.java

1  int[][] tablero = new int[3][4];    // 3 filas, 4 columnas
2  tablero[1][2] = 7;                  // fila 1, columna 2
3
4  // Inicialización directa:
5  int[][] m = {
6      {1, 2, 3},
7      {4, 5, 6}
8  };
9
10 // Recorrer con dos bucles anidados:
11 for (int f = 0; f < m.length; f++) {    // filas
12     for (int c = 0; c < m[f].length; c++) { // columnas de esa fila
13         System.out.print(m[f][c] + " ");
14     }
15     System.out.println();
16 }
```

Consola / Terminal

```
1 2 3
4 5 6
```

1.6 La caja de herramientas: `java.util.Arrays`

La clase `Arrays` trae métodos listos para tareas comunes. Solo necesitas importarla con `import java.util.Arrays;`

Método	Qué hace
<code>Arrays.toString(a)</code>	Convierte el array en un texto legible para imprimirlo.
<code>Arrays.sort(a)</code>	Ordena el array de menor a mayor.

Método	Qué hace
<code>Arrays.fill(a, v)</code>	Rellena todas las casillas con el valor v.
<code>Arrays.copyOf(a, n)</code>	Crea una copia del array con un nuevo tamaño n.
<code>Arrays.equals(a, b)</code>	Compara si dos arrays tienen los mismos elementos.

```
UtilArrays.java

1 import java.util.Arrays;
2
3 int[] a = {5, 2, 8, 1};
4 Arrays.sort(a);
5 System.out.println(Arrays.toString(a)); // [1, 2, 5, 8]
```

2 Strings: el texto en Java

Un String es una secuencia de caracteres: nombres, mensajes, frases. Es una de las clases más usadas de Java, y tiene una característica clave que debes entender bien: es inmutable.

2.1 Crear strings y la inmutabilidad

Inmutable significa que, una vez creado, un String NO se puede cambiar. Cualquier operación que “parezca” modificarlo en realidad crea un String nuevo y deja el original intacto.

```
Strings1.java

1 String saludo = "Hola";           // forma habitual (literal)
2 String otro   = new String("Hola"); // válida pero innecesaria
3
4 saludo = saludo + " mundo";        // NO modifica "Hola":
5                                     // crea un nuevo String "Hola mundo"
6 System.out.println(saludo);        // Hola mundo
```

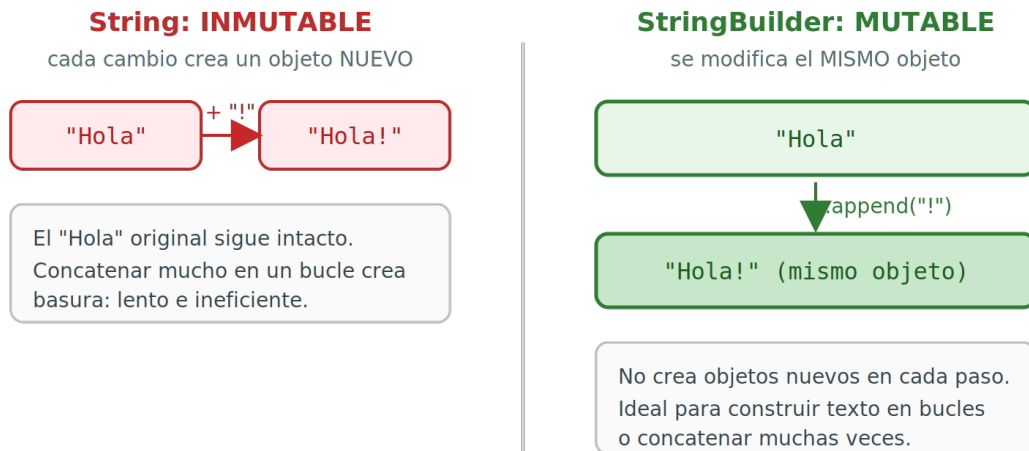


Figura 3.3 — Modificar un String crea objetos nuevos; StringBuilder reutiliza el mismo.

2.2 Métodos comunes de String

Estos son los métodos que usarás a diario. Recuerda: ninguno cambia el String original; todos devuelven un resultado nuevo.

Método	Qué hace	Ejemplo (s = "Java")
<code>length()</code>	Cantidad de caracteres	<code>s.length()</code> → 4
<code>charAt(i)</code>	Carácter en la posición i	<code>s.charAt(0)</code> → 'J'
<code>substring(a,b)</code>	Trozo desde a hasta b (sin incluir b)	<code>s.substring(1,3)</code> → "av"
<code>indexOf(x)</code>	Posición de x (o -1 si no está)	<code>s.indexOf("v")</code> → 2
<code>toUpperCase()</code>	Todo a mayúsculas	<code>s.toUpperCase()</code> → "JAVA"
<code>toLowerCase()</code>	Todo a minúsculas	<code>"JAVA".toLowerCase()</code> → "java"
<code>contains(x)</code>	¿Contiene el texto x?	<code>s.contains("av")</code> → true

Método	Qué hace	Ejemplo (s = "Java")
<code>replace(a,b)</code>	Reemplaza a por b	<code>s.replace("a","o") → "Jovo"</code>
<code>trim()</code>	Quita espacios al inicio y al final	<code>" hi ".trim() → "hi"</code>
<code>equals(x)</code>	¿Mismo contenido? (¡no uses <code>==</code> !)	<code>s.equals("Java") → true</code>
<code>split(x)</code>	Parte el texto en un array	<code>"a,b,c".split(",") → [a,b,c]</code>

```

1 String frase = "Programar en Java";
2
3 System.out.println(frase.length());           // 17
4 System.out.println(frase.toUpperCase());       // PROGRAMAR EN JAVA
5 System.out.println(frase.substring(13));       // Java
6 System.out.println(frase.contains("Java"));    // true
7 System.out.println(frase.replace("a", "@"));   // Progr@m@r en J@v@
8
9 String[] palabras = frase.split(" ");          // [Programar, en, Java]
10 System.out.println(palabras.length);          // 3

```

⚠ Recordatorio clave: `==` vs `.equals()`

Para comparar el CONTENIDO de dos textos usa siempre `.equals()` (o `equalsIgnoreCase()` si no importan mayúsculas). El operador `==` compara si son el mismo objeto en memoria, no su texto: una fuente de bugs muy común.

2.3 Concatenar y el problema de rendimiento

Unir strings con `+` es cómodo, pero como cada unión crea un objeto nuevo, hacerlo miles de veces dentro de un bucle es lento y desperdicia memoria. Para eso existe `StringBuilder`.

```

1 // ❌ Lento: crea un String nuevo en CADA vuelta
2 String resultado = "";
3 for (int i = 0; i < 1000; i++) {
4     resultado += i + ",";
5 }

```

2.4 `StringBuilder`: texto mutable y eficiente

`StringBuilder` es como un `String` editable. Lo construyes paso a paso con `append()` y solo al final lo conviertes en `String` con `toString()`. Es la forma recomendada de construir texto largo o en bucles.

```

1 StringBuilder sb = new StringBuilder();
2 for (int i = 0; i < 1000; i++) {

```

```

3     sb.append(i).append(",");    // modifica el MISMO objeto
4 }
5 String resultado = sb.toString(); // Lo convertimos a String al final
6
7 // Otros métodos útiles:
8 sb.insert(0, "Inicio: ");        // inserta en una posición
9 sb.reverse();                    // invierte el texto
10 sb.length();                    // longitud actual

```

2.5 StringBuilder vs StringBuffer

Son casi idénticos; la diferencia está en la concurrencia (varios hilos a la vez):

	StringBuilder	StringBuffer
Velocidad	Más rápido	Más lento
Seguro entre hilos	No	Sí (sincronizado)
Cuándo usar	Casi siempre (un solo hilo)	Solo si varios hilos lo comparten



Buena práctica 2026

Para concatenaciones simples y puntuales, + está perfecto y es legible. Para construir texto dentro de bucles o por muchas partes, usa StringBuilder. Reserva StringBuffer únicamente para escenarios con múltiples hilos: hoy es poco frecuente.



Java moderno: bloques de texto y format

Desde Java 15, los bloques de texto (text blocks) con tres comillas """...""" permiten escribir varias líneas sin secuencias de escape, ideal para JSON, HTML o SQL.

Y String.format("Hola %s, tienes %d", nombre, edad) o el método formatted() crean textos con plantillas de forma muy legible.

3 Métodos: organiza y reutiliza tu código

Un método es un bloque de código con nombre que realiza una tarea concreta. Te permite escribir la lógica UNA vez y reutilizarla cuantas veces quieras, evitando repetir código y haciendo tus programas más claros y fáciles de mantener.

3.1 Anatomía de un método

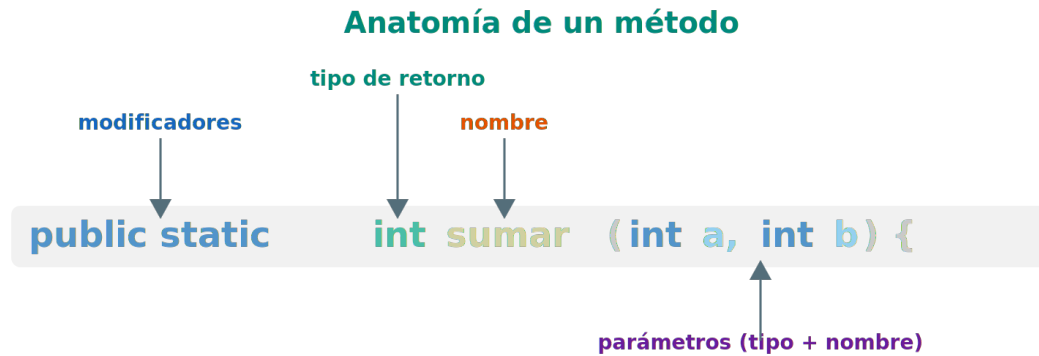


Figura 3.4 — Las partes de un método: modificadores, tipo de retorno, nombre y parámetros.

Cada parte tiene su función:

- Modificadores (public, static...): definen visibilidad y comportamiento.
- Tipo de retorno: el tipo de dato que devuelve. Si no devuelve nada, se usa void.
- Nombre: cómo lo llamarás (por convención, en camelCase y empezando por un verbo).
- Parámetros: los datos de entrada que recibe, cada uno con su tipo y nombre.

3.2 Definir y llamar un método

Defines el método una vez y luego lo “llamas” por su nombre tantas veces como necesites:

```
Metodos1.java

1 public class Calculadora {
2
3     // Definición del método
4     static int sumar(int a, int b) {
5         return a + b;           // devuelve el resultado
6     }
7
8     public static void main(String[] args) {
9         int r = sumar(3, 5);    // Llamada: a=3, b=5
10        System.out.println(r); // 8
11        System.out.println(sumar(10, 20)); // 30 (reutilizado)
12    }
13 }
```

3.3 Parámetros, argumentos y paso por valor

Los parámetros son las variables de entrada que aparecen en la definición; los argumentos son los valores reales que pasas al llamarlo. En Java los primitivos se pasan por valor: el método recibe una COPIA, así que cambiarla no afecta a la variable original.

```
PorValor.java

1 static void intentarCambiar(int x) {
2     x = 99; // solo cambia la copia local
3 }
4
5 public static void main(String[] args) {
6     int numero = 5;
7     intentarCambiar(numero);
8     System.out.println(numero); // 5 -> ¡no cambió!
9 }
```



Matiz importante con objetos y arrays

Con objetos y arrays se pasa por valor la REFERENCIA (la “dirección”). Por eso un método sí puede modificar el contenido de un array que recibe, aunque no puede hacer que la variable original apunte a otro array distinto.

3.4 El tipo de retorno y void

Si un método devuelve un valor, declara su tipo y usa return. Si solo realiza una acción (como imprimir) y no devuelve nada, su tipo es void.

```
Retorno.java

1 // Devuelve un boolean:
2 static boolean esPar(int n) {
3     return n % 2 == 0;
4 }
5
6 // No devuelve nada (void): solo actúa
7 static void saludar(String nombre) {
8     System.out.println("Hola, " + nombre);
9     // no necesita return
10 }
```



Java moderno: varargs

Con los tres puntos (int... numeros) un método puede recibir un número variable de argumentos, que internamente llegan como un array. Así puedes llamar sumar(1, 2, 3) o sumar(1, 2, 3, 4, 5) con el mismo método.

3.5 Sobrecarga de métodos (overloading)

Java permite tener VARIOS métodos con el MISMO nombre, siempre que se diferencien en sus parámetros (en cantidad o tipo). Es la sobrecarga, y deja que un nombre intuitivo sirva para situaciones parecidas. Java elige automáticamente cuál usar según los argumentos.

```

Sobrecarga.java

1 static int    sumar(int a, int b)          { return a + b; }
2 static double sumar(double a, double b)    { return a + b; }
3 static int    sumar(int a, int b, int c)    { return a + b + c; }
4
5 // Java elige el método correcto según los argumentos:
6 sumar(2, 3);           // usa el primero  -> 5
7 sumar(2.5, 3.5);       // usa el segundo  -> 6.0
8 sumar(1, 2, 3);        // usa el tercero  -> 6

```



La firma manda

Lo que distingue a los métodos sobrecargados es su firma: el nombre más la lista de parámetros. Cambiar SOLO el tipo de retorno no basta: dos métodos con igual nombre y parámetros pero distinto retorno NO compilan.

3.6 Recursión: un método que se llama a sí mismo

La recursión ocurre cuando un método se llama a sí mismo para resolver una versión más pequeña del mismo problema. Cada llamada acerca un poco más a un caso base que detiene la cadena. El ejemplo clásico es el factorial ($n! = n \times (n-1) \times \dots \times 1$).

Recursión: factorial(4)

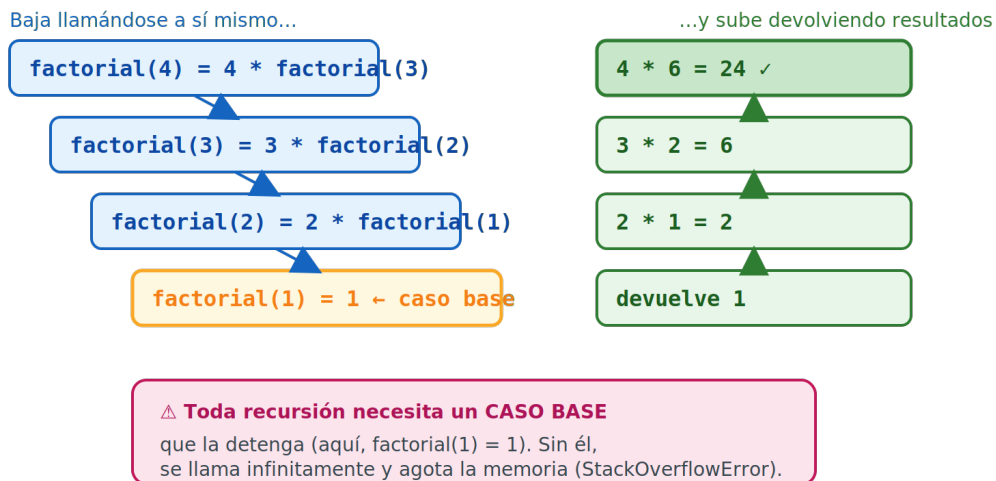


Figura 3.5 — La recursión “baja” llamándose y “sube” devolviendo resultados.

```

Recursion.java

1 static int factorial(int n) {
2     if (n <= 1) {
3         return 1;           // CASO BASE: detiene la recursión
4     }
5     return n * factorial(n - 1); // CASO RECURSIVO
6 }
7
8 System.out.println(factorial(4)); // 4*3*2*1 = 24

```

Sin caso base, desastre

Toda recursión NECESITA un caso base que la detenga. Si falta, el método se llama infinitamente hasta agotar la memoria de la pila y provocar un `StackOverflowError`. Pregúntate siempre: “¿cuándo paro?” y “¿cómo me acerco a ese punto en cada llamada?”.

¿Recursión o bucle?

Todo lo que se hace con recursión puede hacerse con un bucle, y viceversa. La recursión brilla en problemas naturalmente “auto-similares” (árboles, laberintos, divide y vencerás). Para tareas simples, un bucle suele ser más eficiente y fácil de leer.

🚩 Retos del capítulo

Practica combinando arrays, strings y métodos. Intenta resolver cada reto antes de mirar la pista; los últimos integran varios conceptos a la vez.

Reto 1 · El mayor del array [básico]

Escribe un método que reciba un array de enteros y devuelva el valor más grande.

💡 **Pista:** Recorre con `for-each` guardando un máximo provisional.

Reto 2 · Invertir un texto [básico]

Pide una palabra y muéstrala al revés. Hazlo de dos formas: con un bucle y con `StringBuilder`.

💡 **Pista:** `sb.reverse()` resuelve la segunda versión en una línea.

Reto 3 · ¿Es palíndromo? [medio]

Escribe un método boolean que indique si una palabra se lee igual al derecho y al revés (ej: "reconocer").

💡 **Pista:** Compara el texto con su versión invertida usando `equals()`.

Reto 4 · Contador de vocales [medio]

Crea un método que reciba una frase y devuelva cuántas vocales tiene. Ignora mayúsculas/minúsculas.

💡 **Pista:** Convierte a minúsculas y recorre con `charAt(i)`; usa una cadena "aeiou" y `contains()`.

Reto 5 · Sobrecarga de área [medio]

Define un método `area()` sobrecargado: una versión para el área de un cuadrado (un lado) y otra para un rectángulo (base y altura).

💡 **Pista:** Mismo nombre, distinta cantidad de parámetros.

Reto 6 · Fibonacci recursivo (proyecto) [avanzado]

Escribe un método recursivo que devuelva el n-ésimo número de Fibonacci (0,1,1,2,3,5,8...) e imprime los primeros 10 con un bucle. Identifica claramente tu caso base.

💡 **Pista:** $fib(n) = fib(n-1) + fib(n-2)$, con casos base $fib(0)=0$ y $fib(1)=1$.

✓ Resumen del capítulo

Lo esencial que te llevas

- ✓ Un array agrupa valores del mismo tipo; los índices empiezan en 0 y el último es `length - 1`.
- ✓ Las matrices (arrays 2D) se recorren con bucles for anidados y se indexan con `[fila][columna]`.
- ✓ String es inmutable: cada “cambio” crea un objeto nuevo. Compara contenido con `.equals()`, no con `==`.
- ✓ `StringBuilder` construye texto de forma eficiente; `StringBuffer` es su versión segura entre hilos.
- ✓ Un método encapsula una tarea con nombre, parámetros y, opcionalmente, un valor de retorno.
- ✓ La sobrecarga reutiliza un nombre con distintas firmas; los primitivos se pasan por valor (una copia).
- ✓ La recursión resuelve un problema llamándose a sí misma y SIEMPRE necesita un caso base.

En el próximo capítulo: *clases, objetos y los pilares de la programación orientada a objetos.*